

Context-to-Card: A Scenario-Centered Framework for Context-Aware Agent Services

Anonymous Authors

Abstract

Proactive agent services face a decision that ordinary recommendation systems often leave implicit: whether a service should be offered to a particular user at this particular moment. A large set of user, temporal, environmental, event, and place signals does not by itself define a useful service. Directly passing all signals to a text generator can produce content that is weakly related, stale, unsupported, or unnecessarily interruptive. We present **CONTEXT2CARD**, a scenario-centered framework that makes the intermediate reasoning contract explicit. The framework translates structured Context into a bounded situation hypothesis and a latent-need record, performs candidate and fact checks, chooses between a service and an explicit silence outcome, and only then generates a card. We instantiate two card types: an event alert card anchored by a current Event, and a point-of-interest (POI) card that constructs a need before filtering and ranking nearby candidates. Positive and negative synthetic cases are evaluated with a shared rubric covering contextual fit, user value, factual grounding, readability, and restraint. We release a dependency-free reference implementation, schemas, 40 synthetic cases, validators, and an interactive static demo. The released implementation is a transparent research baseline rather than evidence of production or online effectiveness; this distinction is made explicit throughout the paper.

1 Introduction

Recommendation becomes a different problem when the system is allowed to interrupt a user. A conventional recommender can optimize relevance among items. A proactive card service must additionally decide whether the present situation warrants service, whether the facts are still actionable, and whether the output can be traced to evidence. Location and time are known to be important contextual dimensions in recommender systems [1]; however, simply adding more context fields does not explain how an agent should move from observations to a service decision.

We study the following problem. Given a structured Context C , a card type k , and (for POI cards) a candidate set P , produce either a card or a justified silence:

$$f(C, k, P) \rightarrow (d, r, y), \quad d \in \{\text{serve}, \text{stay_silent}\}, \quad (1)$$

where r is an auditable need record and y is card content when $d = \text{serve}$. The goal is not to maximize the number of cards. It is to make the service decision and the generated content appropriate to the current situation, useful for the user, grounded in available facts, and reproducible for evaluation.

Contributions. This paper makes four practical contributions:

1. We define a scenario-centered pipeline in which Context is translated into an auditable **need_record** before candidate selection or language generation.
2. We provide two distinct decision contracts: an Event-anchored alert path and a need-first POI path, while sharing a common service-decision and evaluation layer.
3. We treat **serve** and **stay_silent** as symmetric outputs and provide positive and negative synthetic cases for both card types.
4. We release a transparent reference implementation, schemas, data, validators, and a static demo that expose intermediate outputs rather than only final text.

The release deliberately does not claim improved click-through, user satisfaction, or real-world recommendation quality. Its purpose is to make the reasoning structure and evaluation surface inspectable and replaceable.

2 Related Work

Context-aware recommendation. Context-aware recommender systems incorporate dimensions such as time, location, and social context into recommendation models [1]. The classic pre-filtering, post-filtering, and contextual-modeling perspectives motivate our separation of hard constraints from matching signals. Our focus is complementary: we make the intermediate scenario and service decision explicit for proactive cards, including the option not to recommend.

Agent reasoning and acting. ReAct interleaves reasoning traces and actions to improve interaction with external sources and exception handling [6]. Generative Agents use observations, memory, reflection, and planning to produce believable behavior [5]. We do not attempt to reproduce their general agent capabilities. Instead, we use a small, bounded record that exposes only the evidence and constraints needed for a card-service decision.

Explainability and evaluation. Explainable recommendation research distinguishes user-facing explanations from explanations for system designers and studies their effect on transparency and trust [3, 7]. Our output contract separates short card copy from internal reasons and evidence. Offline evaluation is useful but imperfect for interactive recommenders [2, 4]; consequently, our synthetic results are reported as reproducibility checks, not as estimates of online utility.

3 Problem Formulation and Design Principles

Let $C = (U, E, V, X)$ contain user features U , environment features E , an optional Event V , and an optional POI candidate set X . The framework first constructs a situation hypothesis s and a need record r :

$$(s, r) = g(C, k), \quad r = (\text{situation, need, constraints, evidence, confidence}). \quad (2)$$

The service decision is then computed from the record and relevant facts:

$$d = h(r, V, X). \quad (3)$$

Only if $d = \text{serve}$ is card generation invoked:

$$y = q(r, V, X, k). \quad (4)$$

The separation imposes five design principles:

1. **Evidence before expression:** user-visible claims must be supported by fields in the Case.
2. **Situation before candidate:** a POI is selected to satisfy a translated need, not to justify a preselected item.
3. **Decision before generation:** generation cannot turn a failed service check into a card.
4. **Silence is a result:** no-card outcomes carry a reason and are included in evaluation.
5. **Replaceable modules:** retrieval, ranking, and generation can be replaced without removing the intermediate contract.

4 Framework Architecture

Figure 1 shows the complete data flow. The dashed boundary separates the context-to-card method from the final presentation surface.

4.1 Context roles

Every field is assigned a role before use:

- **Evidence** directly describes a fact, such as an active Event or an open POI.
- **Relation signal** describes why the fact may matter to this user, such as a preference overlap or relevance score.
- **Constraint** limits what can be served, such as distance, time window, weather exposure, or availability.

The same field must not be treated as a universal positive signal. A preference can establish a relation but cannot alone prove an immediate intention to act.

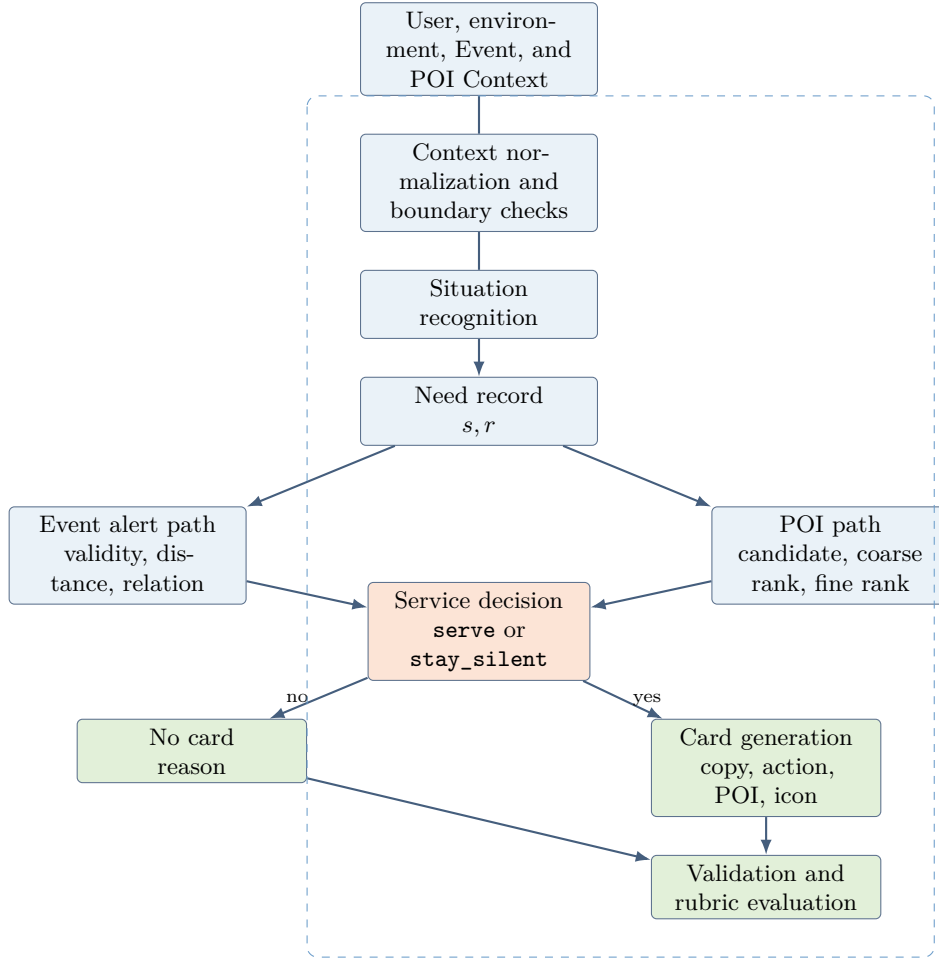


Figure 1: The Context-to-Card architecture. The need record is the explicit interface between context interpretation and service execution.

4.2 The need record as a bounded interface

The need record is intentionally small. It contains a situation statement, a latent need, explicit constraints, supporting evidence, and a confidence value. It is not a user-facing explanation and it is not a free-form chain-of-thought. Downstream components may use only needs and constraints supported by the record. This limits unsupported inference and makes errors localizable.

5 Two Operational Views

The method has a data-construction view and a runtime view (Figure 2). Confusing them creates unrealistic cases: a dataset builder can inspect a global Event or POI table, whereas a runtime component must use only the fields present in the Case.

6 Event Alert Cards

6.1 Contract and anchor

An Event alert card answers whether one current Event is worth surfacing now. The Event is the fact anchor. The public baseline expects a theme, status, distance, user-relevance score, factual line, executable action, and bilingual icon tag. Location-list or ranking content is not treated as an Event in this path; it belongs to the POI path.

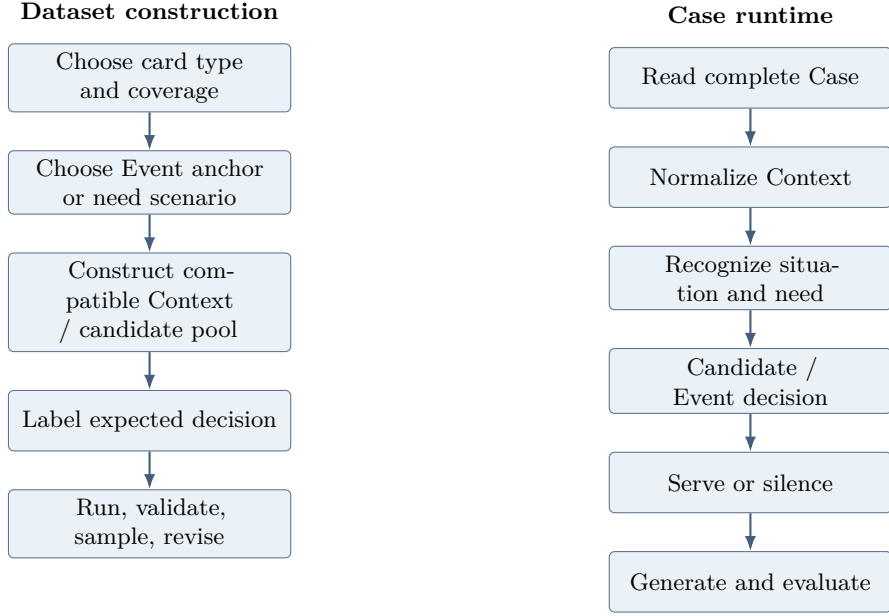


Figure 2: Dataset construction and runtime are related but distinct operational views.

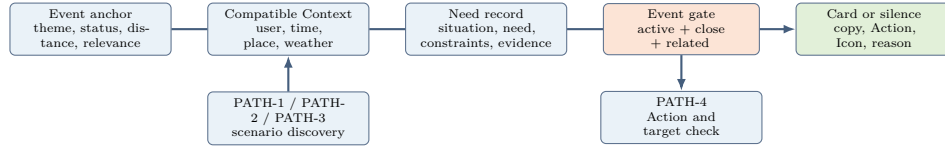


Figure 3: Event alert workflow. Scenario paths discover candidate situations; the Event gate and Action check occur before content generation.

6.2 Scenario-construction paths

Dataset construction can discover an alert scenario through three paths, followed by an action-compatibility check:

PATH-1 (top-down) Start from a broad life domain, independently propose a current opportunity or change, then verify it against an allowed Event.

PATH-2 (seeded expansion) Use an old sub-scenario only as an idea seed; reconstruct it with currently usable Context and an allowed Event. Deprecated dependencies are removed rather than mocked back into existence.

PATH-3 (bottom-up) Combine user relation, a current signal, and an actionable fact from available Context; only then assign the result to a broad domain.

PATH-4 is not a scenario generator. After an alert opportunity is established, it checks whether one available Action naturally completes the service and, for a place action, whether the target POI is traceable. The public reference data does not persist a full construction trace, but the method requires the trace in a production dataset.

6.3 Event–Context compatibility

After selecting an Event, the data builder constructs a compatible user and environment Context. Compatibility is checked across five dimensions: time and Event window, spatial distance, user relation, weather versus indoor/outdoor exposure, and Action support. A missing compatibility fact is not silently filled at runtime; the Case is revised during dataset construction or labeled negative.

6.4 Decision rule

The transparent baseline serves an Event only when:

$$\mathbb{K}[\text{status} = \text{active}] \wedge \mathbb{K}[\text{distance} \leq 8] \wedge \mathbb{K}[\text{relevance} \geq 0.6]. \quad (5)$$

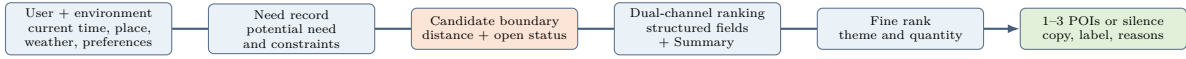


Figure 4: POI workflow.

Otherwise it returns `stay_silent` with one or more reasons. The numerical thresholds are demonstration parameters, not claims about a universal production policy.

6.5 Content contract

The main copy expresses user relation, present value, and the concrete Event. It is limited to 16 Chinese characters in the source product; the public English paper describes this as a bounded short string rather than imposing a language-specific character count. The sub-copy carries a factual line from the Event. One user-facing Action is emitted, using an executable verb phrase rather than an information label. The icon is selected primarily from Event or POI type, with weather as an auxiliary signal, and is emitted in English and Chinese tags.

Worked example. Given a local user with outdoor and low-intensity preferences, a sunny morning, and an active market 1.3 km away, the record may state: "nearby, currently active market; decide whether surfacing it now lowers discovery cost." The decision is `serve`. The card can contain a value-oriented main copy, the factual time-and-place line as sub-copy, the Action "Go see," and the icon tag 'market / market'. If the same Event is expired, the result is `stay_silent` even when distance and preference relation remain high.

7 POI Recommendation Cards

7.1 Need-first candidate construction

POI cards are not interest-only recommendations. They are anchored in the user's current time, place, weather, and usable profile features. The system first translates these signals into a low-decision-cost need and constraints. Distance is the first candidate boundary, followed by availability. Only within that boundary are preference, structured POI attributes, and Summary features compared.

7.2 Candidate pool

Let P be the input candidate pool. The baseline keeps a candidate p only if:

$$\text{distance}(p) \leq 5 \text{ km} \quad \wedge \quad \text{status}(p) = \text{open}. \quad (6)$$

The boundary can be changed by city, mode, time, or user state without changing the ordering principle. A candidate pool is not the final result; it is the evidence set from which the final 1-3 POIs must be selected.

7.3 Coarse ranking with Summary and structured features

The baseline score illustrates the intended two-channel use of POI data:

$$S(p) = 0.45D(p) + 0.35O(p) + 0.20M(p), \quad (7)$$

where D is a normalized distance score, O is preference-tag overlap, and M is a Summary-theme match. In the reference code, the Summary signal is represented by a transparent 'summary_theme' match against POI tags or user tags. More advanced systems may replace this binary feature with semantic matching, but Summary must affect candidate retention or ranking, not merely appear in a post-hoc explanation.

7.4 Fine ranking and quantity

Candidates are sorted by score and distance, with at most three selected. One POI is appropriate when one candidate clearly dominates; two or three are appropriate when candidates share a coherent theme and provide useful choice. The baseline returns `stay_silent` when there is no valid candidate or the highest score is below 0.42. The public baseline does not implement an independent topic-consistency filter; topic consistency is a method requirement and an explicit extension point.

Table 1: Decision contracts for the two card types.

	Event alert	POI recommendation
Fact anchor	One current Event	Candidate POI set
Primary question	Is this Event worth surfacing now?	Which nearby POIs satisfy the current need?
First filters	Status, distance, relevance, time	Distance, open status, need compatibility
Final object	One Event, one Action, one Icon	One to three POIs, theme, copy
Silence examples	Expired, too far, weak relation	No valid candidate, too far, unavailable, weak match

7.5 Content and image-label contract

The internal theme explains why the selected POIs belong together. The main copy communicates current user value and the concrete type of outing; the sub-copy supplies one line of grounded auxiliary facts rather than a generic ETA. Internal recommendation reasons mention distance, preference overlap, Summary evidence, and structured attributes, but are not shown as user copy. A shared image label is selected with the priority "specific secondary tag, then suitable primary category." If neither applies, the output explicitly records that no suitable label was found rather than inventing a universal fallback.

Worked example. For an afternoon user who prefers outdoor, low-intensity activities, the need record can be "find an open, nearby, low-decision-cost outdoor option." Two nearby open POIs whose Summaries describe a greenway walk and an easy cycling route can share the internal theme "nearby light outdoor." A short main copy and a factual sub-copy are generated only after these POIs have been selected. A distant POI with a matching "outdoor" tag is filtered before ranking.

8 Service Decision and Card Generation

The decision layer is shared, while the evidence tested is card-specific:

Generation is invoked only after `serve`. The card output is separated into user-facing fields (main copy, sub-copy, Action or POI name) and internal fields (need record, evidence, theme, decision reasons). This separation allows concise presentation and detailed audit simultaneously.

9 Synthetic Dataset Construction

9.1 Case schema

Each Case contains a unique ID, card type, Context, and expected decision. Running a Case produces an actual decision, need record, card, reasons, and source marker. The reference release has 40 Cases: 10 positive and 10 negative for each card type. Event negatives cover expired status, excessive distance, and low relevance. POI negatives contain only closed or overly distant candidates.

9.2 Coverage and distribution

Coverage is defined over factors that can change a decision or output: time of day, weather, local versus visiting status, preference combinations, Event themes, POI themes, candidate distance, candidate count, ordinary versus special dates, and positive versus negative polarity. The aim is not to enumerate every Cartesian product. Instead, each factor is varied enough to expose a different reasoning boundary, while avoiding a synthetic distribution in which every Case contains a special date or a single repeated template.

9.3 Positive and negative cases

Positive Cases test whether the system serves when facts are current, related, actionable, and sufficiently close. Negative Cases test restraint. A negative result is not a failed generation: it is a valid service outcome with a

Table 2: Public reference artifact checks.

Check	Result	Interpretation
Synthetic Cases	40	10 positive + 10 negative per card type
Decision agreement	40/40	Label and deterministic baseline are consistent
Validator	PASS	Structure and public-boundary checks pass
Unit smoke test	PASS	Both card types and both decisions execute

reason. The two card types have independent negative sets; a POI card does not evade a negative condition by delegating to an Event card.

10 Evaluation Protocol

10.1 Automated checks

The reference validator checks JSON parseability, required top-level keys, card-type and decision enumerations, expected/actual decision agreement, null card content for silence, duplicate IDs, and a small set of public-boundary tokens. It does not prove semantic relevance, factual provenance, Summary quality, or naturalness. Those limitations are part of the release contract.

10.2 Rubric

The shared human/model rubric scores five dimensions from 0 to 2:

1. **D1: contextual fit** — does the output respond to the current situation and user relation?
2. **D2: user value** — does it offer a concrete benefit worth acting on?
3. **D3: factual grounding** — can claims be traced to the Case without fabrication?
4. **D4: readability** — is the output concise, natural, and card-appropriate?
5. **D5: restraint and selection quality** — does the system serve only when justified, and choose appropriate candidates?

A score of 2 means the key judgment and content are strongly aligned; 1 means the direction is usable but has a material weakness; 0 means a key judgment fails. Scores are retained per dimension rather than collapsed into one number so that a fluent but unsupported card is distinguishable from a grounded but awkward one.

10.3 Reference verification results

The public release is intentionally evaluated as a reproducibility artifact. On the 40 deterministic synthetic Cases, the reference generator’s actual decisions agree with the expected decisions for all 40 Cases, and the output validator passes the complete generated file. This is a sanity check of the released pipeline and labels, not a generalization result, user study, or claim of superiority over a recommender baseline. No online intervention, click metric, or real-user data is used.

11 Implementation and Reproducibility

The public implementation is dependency-free Python plus a browser-only static demo. The main entry points are:

- ‘pipeline/engine.py’: need translation, Event decision, POI scoring, and card generation;
- ‘pipeline/build_public_dataset.py’: deterministic synthetic Case construction;
- ‘schemas/’: Context and output schemas;
- ‘validators/validate_outputs.py’: structural and public-boundary checks;
- ‘hf_space/index.html’: interactive visualization of Context, need, candidates, decision, output, and reasons.

The minimal reproduction is:

```
python3 pipeline/run_examples.py
python3 validators/validate_outputs.py examples/generated_outputs.jsonl
python3 -m unittest discover -s tests -v
```

The method described in this paper is broader than the minimal implementation. The baseline represents Summary with a binary theme signal, does not independently enforce multi-POI topic consistency, does not parse detailed Event time windows, and uses time/weather primarily in need construction rather than directly in the POI score. These are transparent extension points, not hidden claims.

12 Limitations, Ethics, and Responsible Use

The dataset is synthetic and cannot establish performance on real users, places, or distributions. Thresholds and weights are illustrative. Automatic checks cover structure and selected hard boundaries but not semantic truth. A real deployment would require authorized data, privacy minimization, security review, calibration against real distributions, human or model evaluation of language, and online safety monitoring.

The framework is designed to reduce unnecessary interruption, but it does not guarantee safe or fair decisions. A preference signal can be stale or incomplete; distance is not the same as accessibility; and a fluent explanation can still mislead if its evidence is wrong. Any deployment should provide a way to suppress or correct recommendations and should treat the public baseline as a research scaffold rather than a ready-made service.

13 Conclusion

We presented CONTEXT2CARD, a scenario-centered framework for proactive Agent cards. Its central idea is to expose the intermediate translation from many Context fields to a bounded need record, then separate candidate processing and service decision from final language generation. Event alert cards and POI recommendation cards share this contract while preserving different fact anchors and selection rules. Positive and negative Cases make both service and restraint measurable, and the released implementation makes the intermediate state inspectable.

The framework is deliberately modular: retrieval, ranking, language generation, and rubric execution can be replaced without abandoning the evidence and decision contracts. Future work should add richer Summary semantics, explicit topic-consistency ranking, complete construction traces, calibrated data distributions, and carefully governed real-world evaluation.

Data and Code Availability

The public code, synthetic data, schemas, validators, and interactive demo are available at:

<https://github.com/Weiran71/scenario-context-agent-cards>
<https://huggingface.co/datasets/LiuXinYan111/scenario-context-agent-cards>
<https://liuxinyan111-scenario-context-agent-cards-demo.static.hf.space/index.html>

References

- [1] Gediminas Adomavicius, Bamshad Mobasher, Francesco Ricci, and Alexander Tuzhilin. Context-aware recommender systems. *AI Magazine*, 32(3):67–80, 2011. doi: 10.1609/aimag.v32i3.2364. URL <https://doi.org/10.1609/aimag.v32i3.2364>.
- [2] Imad Aouali, Amine Benhalloum, Martin Bompaire, Benjamin Heymann, Olivier Jeunen, David Rohde, Otmane Sakhi, and Flavian Vasile. Offline evaluation of reward-optimizing recommender systems: The case of simulation. *arXiv preprint arXiv:2209.08642*, 2022. URL <https://arxiv.org/abs/2209.08642>.
- [3] Xu Chen, Yongfeng Zhang, and Ji-Rong Wen. Measuring "why" in recommender systems: A comprehensive survey on the evaluation of explainable recommendation. *arXiv preprint arXiv:2202.06466*, 2022. URL <https://arxiv.org/abs/2202.06466>.
- [4] Balazs Hidasi and Adam Tibor Czapp. Widespread flaws in offline evaluation of recommender systems. *arXiv preprint arXiv:2307.14951*, 2023. URL <https://arxiv.org/abs/2307.14951>.
- [5] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL <https://arxiv.org/abs/2304.03442>.

- [6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [7] Yongfeng Zhang and Xu Chen. Explainable recommendation: A survey and new perspectives. *arXiv preprint arXiv:1804.11192*, 2018. URL <https://arxiv.org/abs/1804.11192>.